

Análisis Comparativo de Arquitecturas Distribuidas: Microservicios versus Monolito Modular, con Aplicación al Sistema de Control de Laboratorios Informáticos (SCLI)

Isaías Abraham Urbina Romero
Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software
Universidad Técnica Estatal de Quevedo (UTEQ)
Quevedo, Ecuador
Correo: iurbinar@uteq.edu.ec
Repositorio:

<https://github.com/IsaiasUrb/TA-IND-01-Informe-Acad-mico-Individual-An-lisis-Comparativo-de-Arquitecturas-Distribuidas.git>

Abstract—Objetivo: comparar la arquitectura de monolito modular y la arquitectura de microservicios, e identificar qué módulos del sistema SCLI (Sistema de Control de Laboratorios Informáticos), proyecto integrador de la asignatura, se beneficiarían de una migración hacia microservicios.

Método: se realizó una revisión de literatura especializada y un análisis estático de la estructura del código fuente de SCLI (38 controladores, 32 servicios, 33 repositorios y 19 entidades sobre una única base de datos PostgreSQL), evaluando criterios de acoplamiento, despliegue, consistencia de datos y costo operacional.

Resultados: SCLI presenta una organización por capas y por dominio funcional (gestión de usuarios, horarios, laboratorios, asistencia, solicitudes y reportes) que ya exhibe límites de módulo claros, característica típica de un monolito modular bien estructurado. Sin embargo, todos los módulos comparten un único proceso, una sola base de datos y un único pipeline de despliegue.

Conclusión: para el alcance académico del PFC, mantener SCLI como monolito modular resulta más adecuado que fragmentarlo en microservicios, aunque dos módulos (notificaciones por correo y generación de reportes PDF) son candidatos viables para una extracción incremental como servicios independientes.

Index Terms—Arquitectura de software, monolito modular, microservicios, sistemas distribuidos, acoplamiento, SCLI.

Abstract—Objective: to compare modular monolith and microservice architectures and identify which modules of the SCLI system could benefit from migration toward microservices.

Method: a literature review combined with a static analysis of the SCLI source code (38 controllers, 32 services, 33 repositories and 19 entities over a single PostgreSQL database) was performed, evaluating coupling, deployability, data consistency and operational cost.

Results: SCLI already exhibits clear functional module boundaries, a hallmark of a well-structured modular monolith. However, all modules share a single process, database and deployment pipeline.

Conclusion: for the academic scope of the project, keeping SCLI as a modular monolith is more appropriate than fragmenting it into microservices, although the email-notification and

PDF-reporting modules are viable candidates for incremental extraction.

Index Terms—Software architecture, modular monolith, microservices, distributed systems, coupling, SCLI.

I. INTRODUCCIÓN

El diseño de la arquitectura de un sistema de software determina, en gran medida, qué tan fácil será desplegarlo, escalarlo y mantenerlo a lo largo del tiempo. Durante años, la arquitectura monolítica fue la opción predominante para la mayoría de sistemas empresariales. A partir de la década de 2010, la arquitectura de microservicios se popularizó como alternativa, dividiendo el sistema en procesos pequeños e independientes que se comunican mediante la red [1].

Sin embargo, esta dicotomía simple esconde un punto intermedio que ha cobrado relevancia en los últimos años: el monolito modular, un sistema que se despliega como una sola unidad pero cuyo código interno está organizado en módulos con fronteras explícitas y bajo acoplamiento entre ellos. La literatura reciente muestra que la elección entre estas arquitecturas no es trivial ni unidireccional: en pruebas de carga y concurrencia, las arquitecturas monolíticas han mostrado en algunos escenarios un throughput superior al de los microservicios equivalentes [2], mientras que los microservicios aportan ventajas claras en escalabilidad independiente y despliegue continuo cuando el equipo y la organización lo requieren [3].

A. Problema

Los equipos de desarrollo especialmente en proyectos académicos o de pequeña escala suelen enfrentar la decisión de adoptar microservicios desde el inicio, atraídos por su popularidad, sin evaluar si la complejidad operacional adicional (orquestración, comunicación entre procesos, consistencia

eventual, observabilidad distribuida) está justificada por los requisitos reales del sistema. Esta brecha entre la adopción de microservicios y la necesidad real del proyecto ha sido documentada en estudios que advierten sobre los costos de una migración prematura [4].

B. Preguntas de investigación

- RQ1. ¿Qué diferencias arquitectónicas existen entre un monolito modular y una arquitectura de microservicios equivalente?
- RQ2. ¿Qué módulos funcionales de SCLI presentarían un beneficio real si se extrajeran como microservicios independientes?

C. Contribución

Este informe aporta:

- 1) Una comparación conceptual entre monolito modular y microservicios.
- 2) Un análisis estructural del sistema SCLI.
- 3) Una recomendación arquitectónica justificada para el PFC.

El resto del informe se organiza así: la Sección 2 presenta el marco teórico; la Sección 3 describe la arquitectura actual de SCLI; la Sección 4 desarrolla el análisis comparativo; la Sección 5 discute la aplicación al PFC; y la Sección 6 presenta las conclusiones y el trabajo futuro.

II. MARCO TEÓRICO

A. Arquitectura monolítica y monolito modular

Un sistema monolítico se construye y despliega como una sola unidad ejecutable. La interfaz de usuario, la lógica de negocio y el acceso a datos residen en el mismo proceso y comparten una única base de datos [2]. Esta arquitectura simplifica el desarrollo inicial, las pruebas de extremo a extremo y el despliegue, ya que no existen llamadas de red entre componentes internos.

El monolito modular conserva esta unidad de despliegue, pero organiza el código en módulos con responsabilidades bien definidas y fronteras explícitas.

B. Arquitectura de microservicios

Una arquitectura de microservicios descompone el sistema en servicios pequeños, cada uno responsable de una capacidad específica de negocio, ejecutándose en procesos independientes y comunicándose mediante protocolos de red [1]. Cada servicio puede desarrollarse, probarse, desplegarse y escalarse de manera independiente, lo que favorece la entrega continua y la asignación de equipos autónomos por servicio. No obstante, esta independencia tiene un costo: la consistencia de los datos pasa de ser transaccional (una sola base de datos, transacciones ACID) a ser eventual entre servicios; aparece la necesidad de patrones de comunicación, descubrimiento de servicios y tolerancia a fallos de red; y la observabilidad (trazas distribuidas, métricas, logs centralizados) se vuelve indispensable para depurar el sistema [4].

C. Criterios de comparación

Los criterios considerados fueron:

- 1) Acoplamiento y despliegue.
- 2) Gestión y consistencia de datos.
- 3) Escalabilidad.
- 4) Complejidad operacional.

La evidencia empírica indica que, para sistemas con baja concurrencia y equipos pequeños, los costos de la complejidad operacional de los microservicios pueden superar sus beneficios de escalabilidad, mientras que para sistemas con múltiples equipos y necesidades de escalado independiente, ocurre lo contrario [4].

III. DESCRIPCIÓN DEL SISTEMA SCLI

SCLI (Sistema de Control de Laboratorios Informáticos) es una aplicación web desarrollada Spring Boot 3.5.4 sobre Java 17, con vistas renderizadas en el servidor mediante Thymeleaf, persistencia con Spring Data JPA/Hibernate sobre una única base de datos PostgreSQL (BD_Laboratorio1), y generación de reportes en PDF con OpenPDF. El sistema se despliega como un único artefacto ejecutable (.jar) que expone todos sus servicios en el puerto 8081.

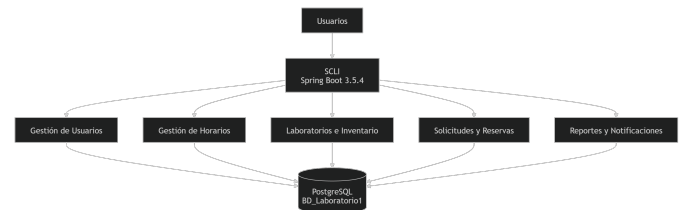


Figura 1. Arquitectura general del Sistema de Control de Laboratorios Informáticos (SCLI).

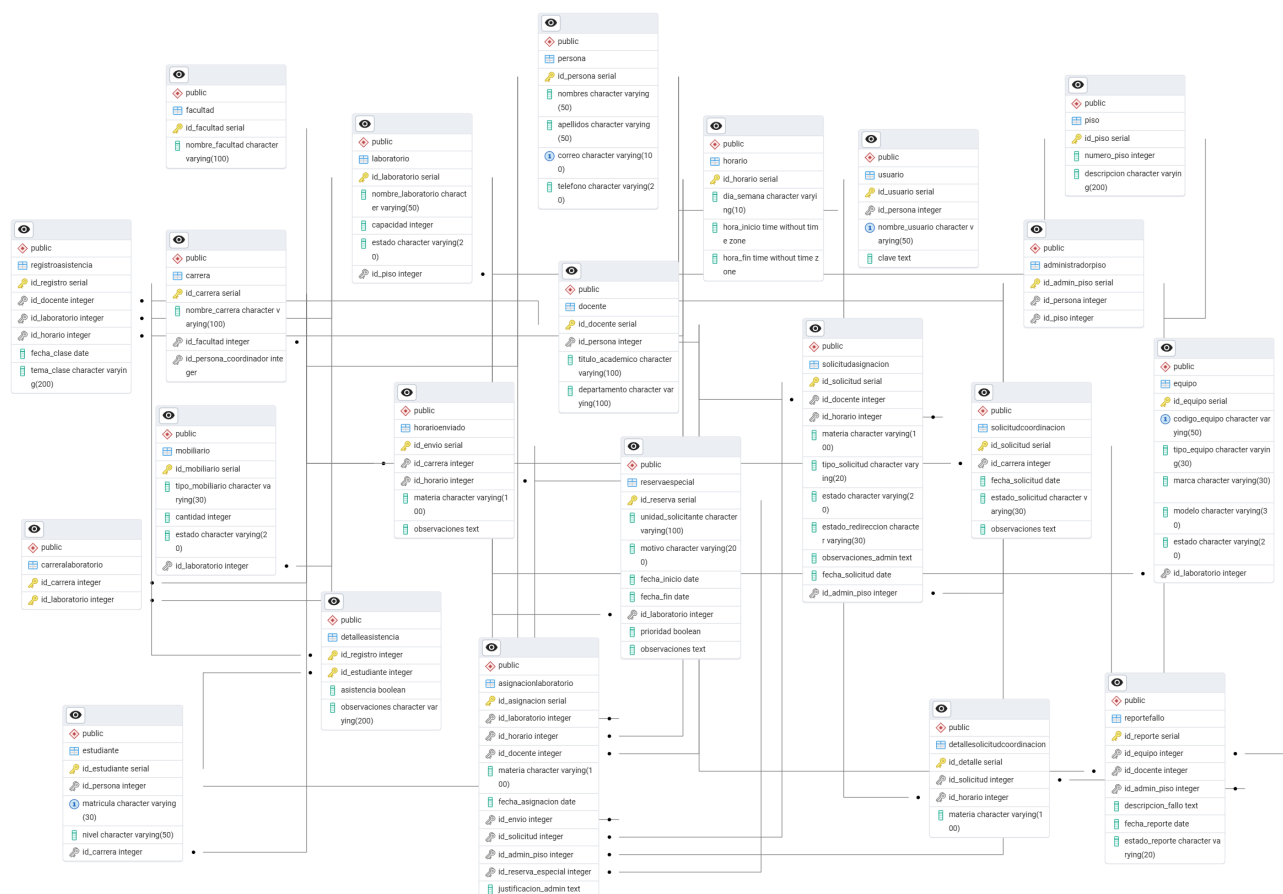
Como se observa en la Figura 1, SCLI está organizado en varios dominios funcionales que comparten una única base de datos PostgreSQL y se ejecutan dentro de una aplicación Spring Boot, característica propia de un monolito modular.

La Figura 2 muestra la estructura lógica de la base de datos utilizada por el sistema. El modelo evidencia una fuerte integración entre los distintos dominios funcionales mediante relaciones que garantizan la integridad referencial y la consistencia de la información almacenada.

A. Organización interna del código

El análisis del repositorio identifica:

- Capa de controladores (38 clases): expone los endpoints web y REST, separados por rol de usuario (estudiante, docente, coordinador, administrador) y por dominio (MateriaController, HorarioApiController, LaboratorioApiController, EquipoApiController, AdminSolicitudesApiController, ReporteController, entre otros).
- Capa de servicios (32 clases): contiene la lógica de negocio, también separada por dominio (UsuarioService,



HorarioDocenteService, AsistenciaService, ReservaEspecialService, EmailService, PdfReport, AuditService, etc.).

La estructura corresponde a un monolito modular organizado por capas y dominios funcionales.

Se identifican seis dominios:

5) Gestión de solicitudes y reservas.

IV. ANÁLISIS COMPARATIVO

V. DISCUSIÓN Y APLICACIÓN AL PFC

Una migración completa no se considera adecuada debido a:

Cuadro I
COMPARACIÓN ENTRE MONOLITO MODULAR Y MICROSERVICIOS

Criterio	Monolito Modular (SCLI)	Microservicios
Unidad de despliegue	Un solo artefacto ejecutable y un único despliegue.	Múltiples servicios independientes con despliegue autónomo.
Comunicación	Llamadas en memoria entre componentes.	Comunicación mediante REST o mensajería.
Gestión de datos	Una única base de datos con integridad referencial.	Bases de datos independientes y consistencia eventual.
Escalabilidad	Escala como una única unidad.	Escalado independiente por servicio.
Complejidad operacional	Baja complejidad de infraestructura.	Mayor complejidad: observabilidad, API Gateway y orquestación.

bases de datos distintas obligaría a reimplementar manualmente controles de integridad que hoy garantiza PostgreSQL.

- 3) Restricciones de tiempo y tamaño del equipo. 1. Construir y mantener un API Gateay, descubrimiento de servicios y observabilidad distribuida para seis servicios excede el tiempo disponible en un período académico, y desplazaría esfuerzo desde las funcionalidades del laboratorio hacia infraestructura.

B. Extracción incremental de servicios

Se identifican dos candidatos:

- Servicio de notificaciones (EmailService). El envío de correos (confirmaciones de solicitudes, avisos) es una operación que no requiere respuesta inmediata ni participa en las transacciones principales sobre BD_Laboratorio1. Podría extraerse como un microservicio que recibe eventos (por ejemplo, “solicitud aprobada”) a través de una cola de mensajes y los procesa de forma asíncrona, ilustrando el patrón de Event-Driven Architecture (EDA).
- Servicio de generación de reportes PDF. La generación de reportes en PDF es una operación intensiva en cómputo que no necesita ejecutarse en el mismo proceso que atiende las peticiones interactivas de los usuarios. Extraerlo como servicio in-dependiente permitiría escalarlo por separado en momentos de alta demanda (por ejemplo, al cierre de un período) y experimentar con un patrón API tateray simple que enrute las solicitudes de reporte hacia este servicio .

Esta estrategia corresponde a lo que la literatura describe como una migración incremental, en la que el núcleo permanece como monolito modular mientras se extraen, uno a uno, los componentes con menor acoplamiento al modelo transaccional central [1], [4]. De este modo, el equipo obtiene experiencia práctica con comunicación entre procesos y arquitecturas distribuidas (alineado con los objetivos OE3 y OE4 de la asignatura) sin arriesgar la estabilidad del núcleo académico (gestión de usuarios, horarios, laboratorios y solicitudes) , que se beneficia de permanecer como monolito modular.

C. Relación con ISO/IEC 25010:2023

Desde la perspectiva de ISO/IEC 25010:2023, mantener el núcleo de SCLI como monolito modular favorece los atributos

de mantenibilidad (módulos cohesivos, bajo acoplamiento interno) y fiabilidad (transacciones atómicas garantizadas por la base de datos única) , mientras que la extracción incremental de los dos servicios propuestos aporta mejoras puntuales en eficiencia de desempeño (escalado independiente del módulo de reportes) y en compatibilidad/portabilidad, al exponer una interfaz de mensajería que podría integrarse con otros sistemas de la facultad en el futuro.

VI. CONCLUSIONES Y TRABAJO FUTURO

A. Conclusiones

- Respecto a RQ1, el análisis comparativo confirma que la diferencia fundamental entre un monolito modular como SCLI y una arquitectura de microservicios equivalente no radica en la organización interna del código que en SCLI ya está modularizada en seis dominios funcionales bien diferenciados sino en la unidad de despliegue, la naturaleza de la co-municación (en memoria frente a red) y el modelo de consistencia de datos (transaccional frente a eventual).
- Respecto a RQ2, de los seis dominios funcionales identificados en SCLI, dos (notificaciones por correo y generación de reportes en PDF) presentan el mejor balance entre beneficio y costo para una extracción como microservicios independientes, mientras que los cuatro dominios restantes (identidad/usuarios, gestión académica, horarios y laboratorios/solicitudes) deberían permanecer en el núcleo monolítico debido a su alto grado de interdependencia transaccional.
- En consecuencia, la recomendación arquitectónica para el PFC es mantener SCLI como monolito modular, reforzando las fronteras entre sus dominios funcionales actuales, y abordar la migración a microservicios únicamente de forma incremental y acotada a los dos servicios identificados, como ejercicio formativo dentro del alcance de las Unidades 3 y 4 de la asignatura.

B. Trabajo futuro

- Documentar mediante diagramas C4 (Nivel 2—3) la frontera propuesta entre el núcleo monolítico de SCLI y los dos servicios candidatos, como insumo para la Entrega 2 del PFC (arquitectura, API REST y diseño Docker).
- Definir el contrato de eventos (formato de mensaje, cola a utilizar) entre el núcleo de SCLI y el servicio

de notificaciones, especificando qué evento de negocio dispara cada notificación.

- Diseñar el contrato OpenAPI 3.0 del servicio de reportes y un API tateray mínimo que enrute las peticiones /reportes/* hacia dicho servicio, conservando el resto de rutas en el monolito.
- En una fase posterior (Entrega 3), implementar y medir empíricamente con al menos 5 repeticiones y estadística descriptiva, conforme a la rúbrica de la asignatura el tiempo de respuesta de la generación de reportes antes y después de la extracción, para verificar si las mejoras de eficiencia anticipadas en la Sección 5.3 se cumplen en la práctica.

REFERENCIAS

- [1] S. Newman, *Building Microservices, 2nd Edition*, O'Reilly Media, 2021.
- [2] O. Al-Debagy and P. Martinek, "A Comparative Review of Microservices and Monolithic Architectures," *IEEE CINTI*, pp. 149–154, 2018.
- [3] G. Blinowski, A. Ojdowska and A. Przybylek, "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," *IEEE Access*, vol. 10, pp. 20357–20374, 2022.
- [4] I. Karabey Aksakalli, T. Çelik, A. B. Can and B. Tekinerdogan, "Deployment and Communication Patterns in Microservice Architectures," *Journal of Systems and Software*, vol. 180, 2021.